

UNIDAD 11:

ENTRENAMIENTO POR REFUERZO

CURSO

IA

```
01 import openai
02 from langchain import
03   ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16 if __name__ == "__main__":
17     prompt = "Explicame qué
18     es la inteligencia artificial"
19     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com

Tel: +54 351 3733383



TIPOS DE ENTRENAMIENTO EN INTELIGENCIA ARTIFICIAL

- En esta unidad, abordaremos los distintos enfoques mediante los cuales se entrena un sistema de inteligencia artificial.
- Estos métodos definen cómo una IA aprende a partir de datos y cómo ajusta sus parámetros internos para tomar decisiones o realizar predicciones.
- Comprender las distintas formas de entrenamiento es esencial para diseñar modelos adecuados a distintos tipos de problemas y conjuntos de datos.

A continuación, se describen los principales tipos de entrenamiento en inteligencia artificial:

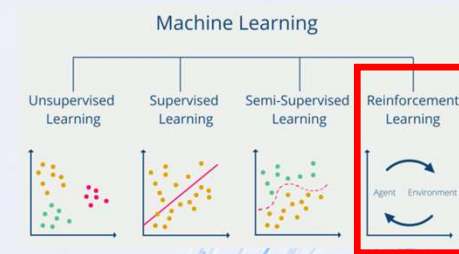
Tipo de Entrenamiento	Definición	Características	Ejemplos
Supervisado	El modelo aprende a partir de datos etiquetados, asociando entradas con salidas correctas.	- Requiere datos etiquetados. - Se usa en clasificación y regresión. - Permite evaluar precisión directamente.	- Clasificación de correos como spam. - Predicción de precios de viviendas.
Por Refuerzo	Un agente aprende interactuando con un entorno, tomando decisiones y recibiendo recompensas o penalizaciones.	- Aprende por experiencia. - Útil en entornos dinámicos. - Requiere balance entre exploración y explotación.	- Juegos como ajedrez. - Control de robots.
No Supervisado	El modelo trabaja con datos sin etiquetas, buscando patrones o agrupamientos ocultos.	- No necesita etiquetas. - Se aplica en agrupamiento y reducción de dimensionalidad. - Evaluación subjetiva.	- Segmentación de clientes. - Análisis de temas en textos.
Semi-supervisado	Se entrena con una pequeña cantidad de datos etiquetados y una gran cantidad sin etiquetar.	- Mezcla supervisado y no supervisado. - Reduce costo de etiquetado. - Usa técnicas como autoetiquetado.	- Clasificación con pocos ejemplos. - Detección de fraude con escasos datos confirmados.
Auto-supervisado	El sistema genera sus propias etiquetas a partir de los datos, usando tareas auxiliares para aprender representaciones útiles.	- No requiere etiquetas humanas. - Escalable a grandes volúmenes. - Muy utilizado en lenguaje natural y visión artificial.	- Predicción de palabras en texto. - Reconstrucción de imágenes incompletas.



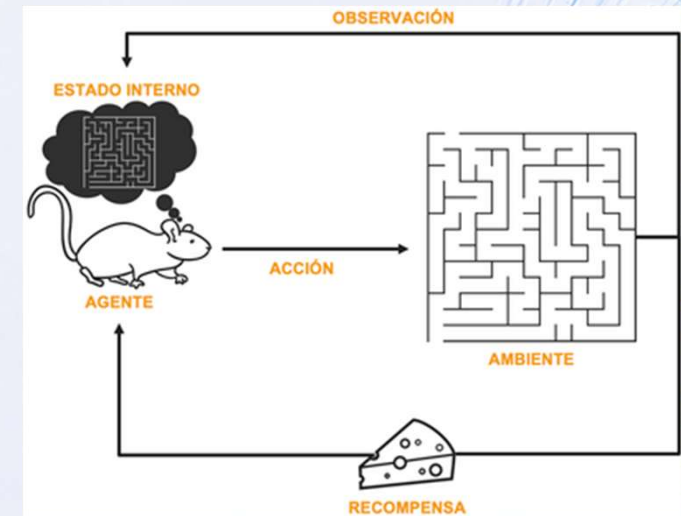
ENTRENAMIENTO POR REFUERZO

El **entrenamiento por refuerzo** es una forma de aprendizaje automático basada en la **interacción con un entorno**.

En lugar de aprender a partir de ejemplos etiquetados (como en el aprendizaje supervisado), un agente **aprende mediante prueba y error**, tomando decisiones y recibiendo **recompensas o penalizaciones** según los resultados de sus acciones.



Aspecto	Descripción
Definición	Es un tipo de aprendizaje automático en el que un agente aprende a tomar decisiones mediante la interacción con un entorno, recibiendo recompensas o penalizaciones según sus acciones.
Funcionamiento	1. El agente observa el estado del entorno. 2. Toma una acción según su política. 3. Recibe una recompensa y un nuevo estado. 4. Ajusta su estrategia para maximizar las recompensas acumuladas.
Usos principales	- Tareas secuenciales con consecuencias a largo plazo. - Sistemas donde no hay etiquetas disponibles, pero sí retroalimentación tras cada acción.
Ejemplos	- Juegos (ajedrez, Go, videojuegos) - Robots que aprenden a caminar o evitar obstáculos - Sistemas de recomendación que aprenden de la interacción del usuario - Control de tráfico inteligente
Requisitos	- Un entorno simulado o real para interactuar - Definición clara de recompensas - Capacidad de experimentar muchas veces (interacción repetitiva)
Ventajas	- Aprende sin datos etiquetados - Ideal para tareas donde una secuencia de decisiones influye en el resultado - Puede alcanzar comportamientos complejos a través de prueba y error
Desventajas	- Requiere muchas interacciones para aprender correctamente - El entrenamiento puede ser inestable o lento - Riesgo de explorar poco o demasiado (dilema exploración-explotación)

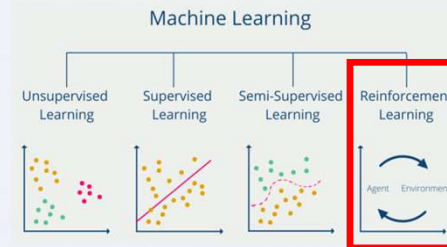




ENTRENAMIENTO POR REFUERZO

Conceptos Nuevos:

Concepto nuevo en RL	Por qué no existía en Supervisado
Agente	En supervisado no hay un actor que decide y actúa; sólo hay un modelo que predice sobre datos ya dados.
Entorno (Environment)	No existe la noción de un "mundo" simulado que responde dinámicamente a las acciones.
Estado (state, s)	Reemplaza al vector de entrada X, pero cambia en el tiempo como consecuencia de la acción anterior — en supervisado los ejemplos son independientes entre sí.
Acción (action, a)	No hay equivalente: el modelo supervisado nunca "actúa", sólo predice.
Recompensa (reward, r)	Reemplaza a la etiqueta Y, pero es una señal de refuerzo, no la respuesta correcta. Puede ser escasa, retardada y ruidosa.
Política (policy, π)	Es la función que el agente aprende: "qué hacer dado un estado". No es un simple mapeo entrada→etiqueta, sino entrada→decisión.
Exploración vs. Explotación	Dilema inexistente en supervisado: el agente debe decidir entre probar acciones nuevas o repetir lo que ya sabe que funciona.
Replay Buffer	El "dataset" ya no es fijo: se construye dinámicamente con la propia experiencia del agente y se va renovando.
Retroalimentación retardada	La consecuencia de una acción (por ejemplo, derribar al enemigo) puede llegar muchos pasos después de haberla ejecutado — no hay "etiqueta inmediata" por ejemplo.
Naturaleza secuencial del problema	El orden de las decisiones importa: una acción afecta el estado futuro, cosa que no ocurre entre ejemplos independientes de un dataset supervisado.



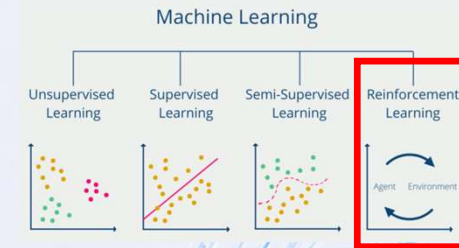
COMPARACIÓN DE CONCEPTOS: APRENDIZAJE SUPERVISADO vs. POR REFUERZO

APRENDIZAJE SUPERVISADO		APRENDIZAJE POR REFUERZO
MODELO: Predice sobre datos dados ($X \rightarrow Y$). No decide.	QUIÉN ACTÚA	AGENTE: Un actor que decide y actúa en el entorno.
DATASET ESTÁTICO: No responde a acciones.	EL MUNDO	ENTORNO (Environment): Un mundo simulado y dinámico.
INMEDIATA: La respuesta correcta para cada ejemplo.	RETROALIMENTACIÓN	RECOMPENSA (Reward, r): Señal indirecta, acumulativa y retardada.
MINIMIZAR ERROR: Ajustar predicciones a etiquetas fijas.	APRENDIZAJE	MAXIMIZAR RECOMPENSA: Aprender una POLÍTICA (π) de decisión secuencial.
SIN DILEMA: Usar datos dados. No hay exploración.	DILEMA	EXPLORACIÓN VS. EXPLOTACIÓN: Probar o repetir acciones.
DATASET FIJO: Ejemplos independientes entre sí.	DATOS	BUFFER DINÁMICO: Construido secuencialmente con experiencia.

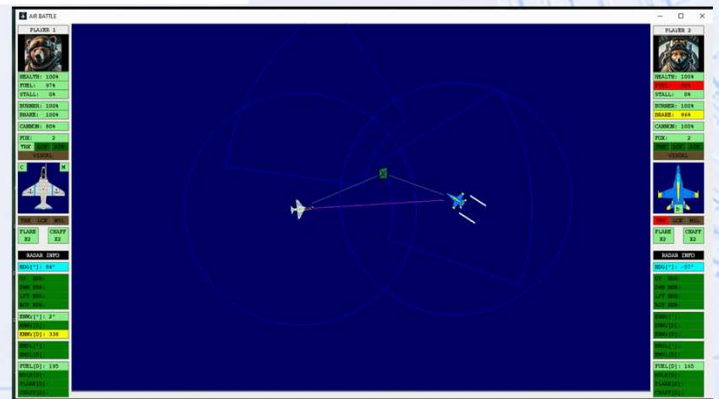
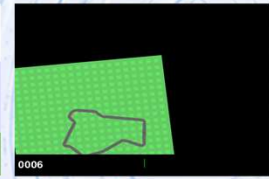
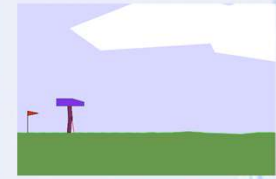
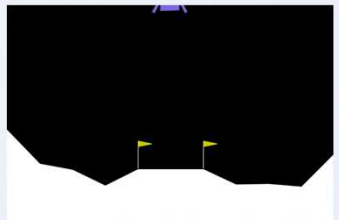


ENTRENAMIENTO POR REFUERZO [ETAPA 1]

Etapa	Objetivo	Tareas específicas	Herramientas	Errores frecuentes
1. Definición del entorno	Establecer las reglas del mundo donde el agente actuará.	- Definir estados posibles, recompensas y transiciones. - Programar simulador o entorno realista.	OpenAI Gym, Unity ML-Agents, entornos personalizados	Entornos demasiado simples o irreales, recompensas mal definidas



Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org/ https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io/ https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, supervivencia.	





[ETAPA 1] Definición del entorno

El **entorno** es la representación del mundo donde el agente interactúa. En tu caso, es el juego de combate de aviones visto desde arriba.

El entorno expone **estados** al agente, que incluyen:

- Sensores propios del avión (health, fuel, posición, misiles activos, etc.)
- Sensores del enemigo (health, posición relativa, misiles, items)
- Distancias a bordes y objetos.

El entorno define **acciones** posibles:

* Combinación de teclas binaria (CMD_LEFT, CMD_RIGHT, CMD_BURNER, CMD_BRAKE, CMD_CANNON, CMD_FOX, CMD_FLARE, CMD_CHAFF).

También maneja **recompensas**, calculadas dinámicamente según el efecto de las acciones:

- Daño al enemigo (+) / propio (-)
- Acercamiento estratégico al enemigo (+)
- Recolección de items (+)
- Evitación de bordes y obstáculos (-)
- Bloqueo de misiles enemigos (+)
- Lanzamiento de misiles con lock activo (+)

Área	Aplicación específica	Qué resuelve el RL	Ejemplos / Actores
Robótica Industrial y Locomoción	Robots humanoides y cuadrúpedos	El agente aprende equilibrio, caminata (locomotion) y agarre de objetos (manipulation) en simuladores físicos (MuJoCo, Isaac Gym) antes de pasar al mundo real (Sim-to-Real)	Tesla (Optimus), Boston Dynamics, 1X
	Brazos robóticos en fábricas	Agarre inteligente de objetos de formas distintas (bin-picking), ensamblaje de precisión, control de calidad en líneas de producción	Automatización industrial
	Robótica agrícola	Navegación autónoma entre cultivos y recolección de frutas delicadas sin dañarlas	Cosecha automatizada
Grandes Modelos de Lenguaje (LLMs)	RLHF (Reinforcement Learning from Human Feedback)	Alinea el modelo de lenguaje: el agente es la IA, el estado es la conversación, y evaluadores humanos (o un modelo de preferencia) otorgan la recompensa para lograr respuestas seguras, útiles y coherentes	ChatGPT, Claude, Llama
	Modelos de razonamiento	Se premian las cadenas de razonamiento paso a paso que llegan a la respuesta correcta, aplicado especialmente en generación de código y matemática	Modelos de "reasoning"
Finanzas, Trading y Market Making	Market making y liquidez	Algoritmos que deciden en milisegundos a qué precio colocar órdenes de compra/venta para ganar el spread sin exponerse a caídas bruscas	Trading algorítmico
	Gestión y rebalanceo de carteras	El agente ajusta la proporción de activos (acciones, bonos, divisas) maximizando el retorno y penalizando la volatilidad a largo plazo	Gestión de portafolios
	Detección dinámica de fraude	El sistema aprende a bloquear transacciones ajustando reglas según el comportamiento evolutivo del cliente	Sistemas antifraude bancarios
Gestión de Redes y Energía	Refrigeración e infraestructura	Control de ventiladores, bombas de agua y climatización para minimizar el consumo energético en centros de datos y fábricas	Data centers a gran escala
	Redes eléctricas e hidrógeno (Smart Grids)	Control de baterías para decidir cuándo cargar (energía solar/eólica barata) y cuándo descargar a la red (energía cara)	Redes eléctricas inteligentes
Logística, Supply Chain y Vehículos Autónomos	Control de flotas y rutas	Asignación de paquetes, tiempos de recarga de camiones eléctricos y rutas óptimas ante imprevistos de tráfico	Empresas de logística y delivery
	Vehículos autónomos	El módulo de planificación de conducción (cambiar de carril, incorporarse a una rotonda, frenar suavemente) se entrena con RL, mientras la percepción del entorno usa visión supervisada	Tesla, Waymo, Pony.ai



[ETAPA 2] Inicialización del agente >>> DQN (Deep Q-Network)

El **agente** es quien aprende a decidir qué acciones tomar según el estado actual. Se define un **modelo de política** (por ejemplo, un DQN con red neuronal) que recibe como entrada el vector de sensores y produce un vector de salida binario correspondiente a las acciones.

El agente puede iniciar **aleatoriamente** o cargar un modelo preentrenado.

En **entrenamiento por refuerzo (RL)**, un **modelo de política** es una función que **decide qué acción tomar dado un estado** del entorno.

Matemáticamente:

$$\pi(a|s) = P(\text{acción } a \mid \text{estado } s)$$

La política puede ser:

- **Determinista** (una acción por estado > Feed forward de una DNN.)
- **Estocástica** (probabilidades de acciones).

Su objetivo: **maximizar la recompensa acumulada** a lo largo del tiempo, no solo predecir un valor de salida inmediato.

Ejemplo:

En tu juego de aviones, el estado es un vector de 80 sensores, y la política decide los 8 comandos binarios que debe ejecutar el avión.





[ETAPA 2] Inicialización del agente >>> DQN (Deep Q-Network)

DQN (Deep Q-Network)

Un **DQN** es un tipo de **modelo de política basado en Q-Learning con redes neuronales profundas**.

- Q-Learning aprende la **función de valor $Q(s,a)$** , que estima **cuánto “vale” realizar la acción a en el estado s** :

$$Q(s, a) \approx \text{recompensa acumulada esperada si tomo } a \text{ en } s$$

- En un **DQN**, la función Q se aproxima usando una **red neuronal profunda**:

$$Q_{\theta}(s, a) \approx \text{red neuronal}(s)$$

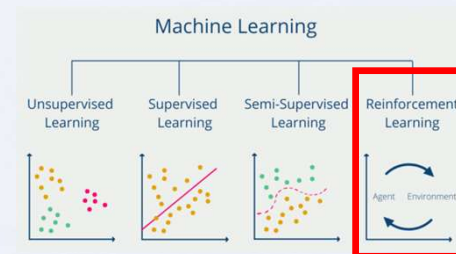
- La red toma el estado como entrada y devuelve un **vector de valores Q** , uno por cada acción posible.

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

- Durante entrenamiento, se actualizan los pesos para que los Q -values se acerquen a las recompensas observadas mediante **temporal difference (TD)**:

$$\pi(s) = \arg \max_a Q(s, a)$$

- La política derivada de un DQN es **“tomar la acción con el Q más alto”**:



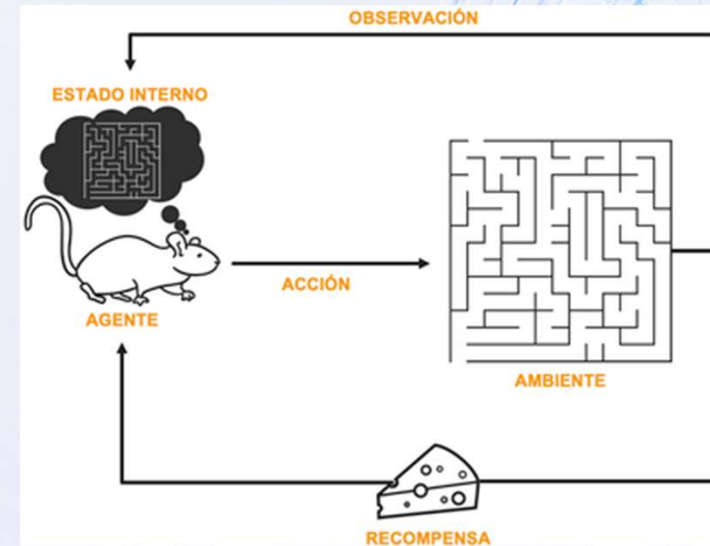
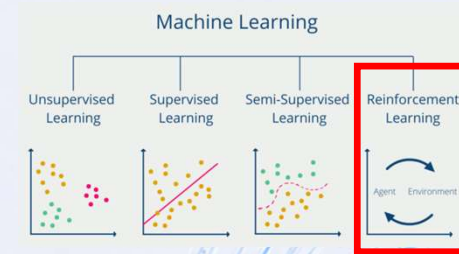
Característica	Red neuronal clásica	DQN / Modelo de política RL
Objetivo	Predecir un valor o etiqueta fija a partir de datos etiquetados	Aprender una política para maximizar recompensas a lo largo del tiempo
Datos de entrada	Dataset $X \rightarrow Y$ con pares fijos	Experiencias (estado, acción, recompensa, siguiente_estado) generadas por interacción con el entorno
Supervisión	Supervisada: existe “respuesta correcta”	Retroalimentación basada en recompensa , que puede ser retardada y no inmediata
Salida	Etiqueta o valor	Q -values o probabilidades de acciones, para decidir qué hacer
Actualización	Gradient descent sobre loss directo	TD-error o política gradient, usa exploración-explotación y replay buffer



[ETAPA 2] Inicialización del agente (PARAMETROS)

Se definen **parámetros de aprendizaje**:

- Tasa de aprendizaje (learning_rate)
- Factor de descuento (gamma)
- Tamaño de la memoria de replay (buffer_size)
- Estrategia de exploración/explotación (epsilon-greedy)





[ETAPA 2] PARAMETROS: Tasa de aprendizaje (learning_rate, α)

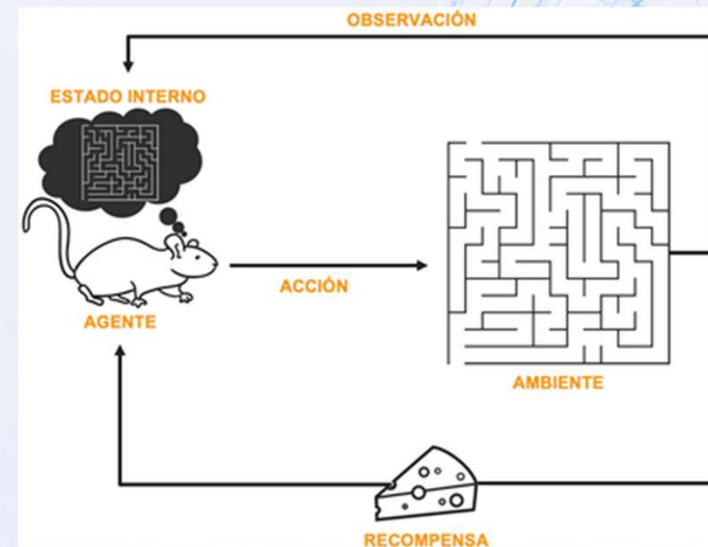
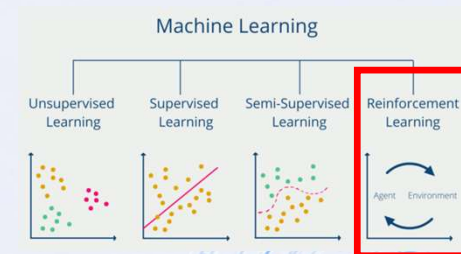
Define **qué tanto se actualizan los pesos de la red neuronal** en cada paso de entrenamiento.

Matemáticamente, en un DQN:

$$Q_{\text{nuevo}} = Q_{\text{viejo}} + \alpha \cdot (\text{target} - Q_{\text{viejo}})$$

- **α pequeño** → aprendizaje más lento pero más estable, evita saltos grandes.
- **α grande** → aprendizaje rápido, pero puede ser inestable o no converger.

Analogía: Es como ajustar el volumen de cada corrección: muy alto puede “sobrepasar”, muy bajo tarda en notar cambios.





[ETAPA 2] PARAMETROS: Factor de descuento (gamma, γ)

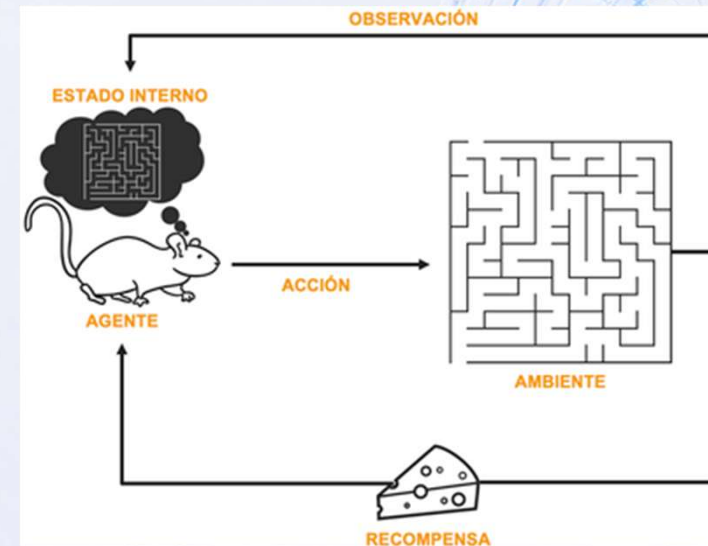
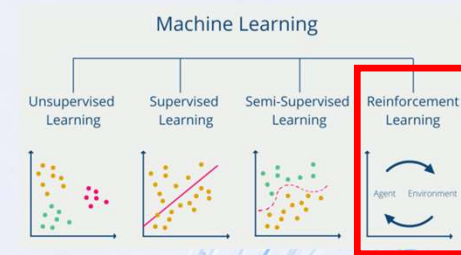
Determina **cuánto valora el agente las recompensas futuras** frente a las inmediatas.

Matemáticamente, la recompensa acumulada se calcula como:

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- **γ cercano a 1** → el agente mira **mucho hacia el futuro**, buscando recompensas a largo plazo.
- **γ cercano a 0** → el agente solo se preocupa por **recompensas inmediatas**, ignorando consecuencias futuras.

Analogía: Es como pensar en el largo plazo vs corto plazo en la toma de decisiones.





[ETAPA 3] Observación del estado

En cada paso de simulación, el agente recibe del entorno el estado actual, que incluye:

- Sensores propios y enemigos
- Distancias a ítems y bordes
- Información de bloqueo y misiles en aire

Este vector de observación representa la percepción del mundo del agente.

PARAMETRO: Tamaño de la memoria de replay [buffer_size]

En DQN se guarda un **replay buffer**, que almacena experiencias.

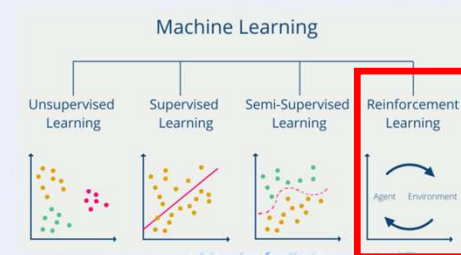
En **reinforcement learning**, una **experiencia** es una unidad de información que representa un paso completo de interacción del agente con el entorno:

Experiencia: (estado, acción, recompensa, siguiente_estado, done)

buffer_size define **cuántas experiencias se pueden almacenar**:

- **Buffer grande** → más diversidad de experiencias, reduce correlaciones y mejora estabilidad, pero requiere más memoria.
- **Buffer pequeño** → se reciclan experiencias rápido, puede generar **overfitting** a pasos recientes y aprendizaje menos estable.

Analogía: Es como tener un cuaderno donde anotas todas tus partidas pasadas: mientras más hojas tengas, mejor puedes aprender de la experiencia global, no solo de lo último que pasó.



EXPERIENCIA	
Elemento	Significado
s (estado)	La observación actual del entorno antes de tomar la acción. En tu juego serían los 40 sensores propios + 40 sensores del enemigo.
a (acción)	La acción que el agente decidió tomar en ese estado. En tu caso, un vector binario de 8 comandos.
r (recompensa)	El valor numérico que indica qué tan buena o mala fue la acción tomada. Puede incluir daño al enemigo, pérdida de combustible, bloqueo de misiles, etc.
s' (siguiente estado)	La observación que el agente recibe después de ejecutar la acción.
done	Booleano que indica si el episodio terminó (por ejemplo, si el avión propio o enemigo fue destruido o se alcanzó el límite de pasos).



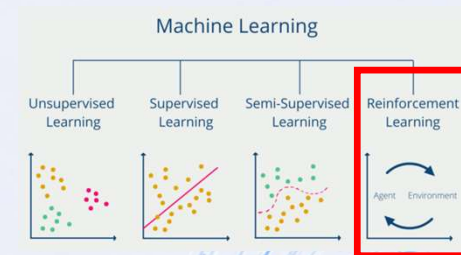
[ETAPA 3] PARAMETROS: Tamaño de la memoria de replay (buffer_size)

Similitudes con un dataset

- Contiene **todas las experiencias del agente** hasta el momento (estado, acción, recompensa, siguiente estado).
- El entrenamiento de la red neuronal se hace **muestreando batches de este buffer**, como en supervisado donde se toman batches del dataset.
- buffer_size es el **tamaño máximo del dataset dinámico**.

Analogía práctica

- Imaginá que tu buffer de 10.000 experiencias es **tu dataset RL**.
- Cada actualización de la red usa un **batch de 64 experiencias aleatorias** → como si tomaras 64 ejemplos de un dataset grande para hacer un paso de entrenamiento supervisado.
- A medida que juegas más partidas, el buffer **se va renovando**, reemplazando las experiencias más antiguas → tu “dataset” **siempre contiene experiencias recientes y relevantes**.



Diferencias con un dataset clásico		
Característica	Buffer RL	Dataset supervisado
Dinámico	Sí, se van agregando experiencias nuevas y descartando las más viejas	Normalmente estático
Orden de muestras	Aleatorio, para romper correlación temporal	Puede ser secuencial o aleatorio
Contenido	Experiencias (s,a,r,s',done)	Ejemplos (X,Y) etiquetados
Tamaño máximo	buffer_size	Depende del dataset original



[ETAPA 4] Selección de acción

Con base en el estado observado:

El agente decide qué acción ejecutar según su **política actual**:

- **Exploración**: probar acciones nuevas al azar (epsilon-greedy)
- **Explotación**: seleccionar la acción con mayor valor esperado según la red neuronal (predicción DQN)

Una **política estocástica** asigna **probabilidades a cada acción** en un estado dado:

$$\pi(a|s) = P(\text{tomar acción } a \text{ dado el estado } s)$$

Esto permite **exploración natural**, porque el agente puede variar sus acciones incluso en el mismo estado.

- **Ventaja**: útil en entornos **complejos o inciertos**, donde tomar siempre la misma acción no es óptimo.
- **Desventaja**: puede ser menos predecible y más difícil de entrenar inicialmente.

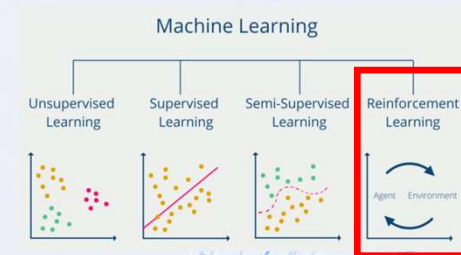
Una **política determinista** elige **una única acción específica** para cada estado.

Matemáticamente:

$$\pi(s) = a$$

Esto significa que **si el agente ve el mismo estado s siempre hará la misma acción a**.

- Ventaja: simple, predecible, funciona bien cuando el entorno es **determinista** o casi determinista.
- Desventaja: puede quedarse atrapado en comportamientos repetitivos y **no explorar otras acciones que podrían ser mejores**.





[ETAPA 4] PARAMETROS: Estrategia de exploración/explotación (epsilon-greedy, ϵ)

Decide **cuándo el agente explora acciones nuevas** vs **cuándo explota su política actual**.
epsilon es la probabilidad de **exploración aleatoria**.

ϵ **alto** → muchas acciones aleatorias → explora mucho, aprende sobre todo el entorno.

ϵ **bajo** → sigue más la política aprendida → se concentra en explotar lo que ya sabe.

Se suele **decrementar con el tiempo**, empezando con mucha exploración y luego volviéndose más determinista.

Analogía: Es como jugar un videojuego: al principio pruebas muchas estrategias nuevas (exploración), luego te concentras en las que sabes que funcionan (explotación).

Función epsilon-greedy:

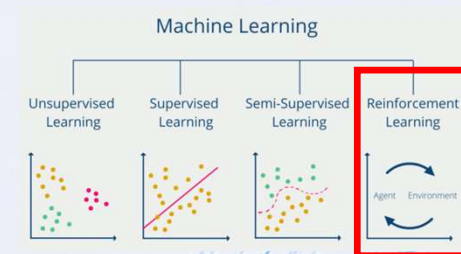
epsilon-greedy es una **estrategia de selección de acción** que mezcla **exploración** y **explotación**:

- **Explotación:** usar la política aprendida para elegir la mejor acción conocida (determinista).
- **Exploración:** elegir una acción **aleatoria** para descubrir nuevas estrategias que podrían ser mejores.

Matemáticamente:

$$\text{acción} = \begin{cases} \text{aleatoria,} & \text{con probabilidad } \epsilon \\ \arg \max_a Q(s, a), & \text{con probabilidad } 1 - \epsilon \end{cases}$$

- epsilon es un número entre 0 y 1 que define la **probabilidad de explorar**.
- Durante el entrenamiento, epsilon suele **decrecer** con el tiempo: se explora mucho al principio y luego se explota más la política aprendida.





[ETAPA 5] Ejecución y retroalimentación

- El entorno aplica la acción al juego real a través del **socket TCP**.
- El juego devuelve el **nuevo estado** y, a partir de este y del estado anterior, se calcula la **recompensa**:

En **reinforcement learning**, la **recompensa (reward)** es un valor numérico que indica **qué tan buena o mala fue la acción del agente en un estado dado**. Es el principal mecanismo de **retroalimentación** que guía al agente a aprender comportamientos deseados.

RECOMPENSA (Reward:)

Para cada paso t en un episodio:

$$r_t = R(s_t, a_t, s_{t+1})$$

s_t = estado actual

a_t = acción tomada

s_{t+1} = siguiente estado

r_t = recompensa asignada por el entorno

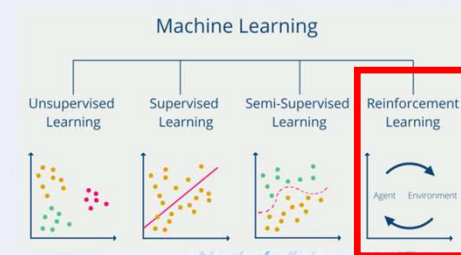
El agente **intenta maximizar la suma de recompensas acumuladas**, considerando tanto el corto como el largo plazo:

$$R_{\text{total}} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

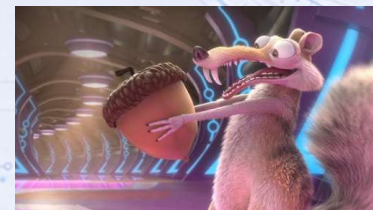
donde γ es el **factor de descuento**.

Principios generales

- Señal positiva** → acciones deseadas
- Señal negativa** → acciones no deseadas o riesgosas
- Balance** → evitar que una sola acción domine la función de recompensa
- Escala consistente** → todas las recompensas deberían estar en rangos comparables para estabilizar el aprendizaje



Componente	Cómo se calcula	Propósito
Daño al enemigo	$\Delta \text{enemy_health} = \text{prev} - \text{actual}$	Incentiva atacar al enemigo
Seguridad propia	$-\Delta \text{self_health}$	Penaliza recibir daño
Distancia al enemigo	$-\text{enemy_dist}$	Incentiva acercarse para atacar
Distancia a ítems	$-\text{item_dist} * 0.5$	Incentiva recoger combustible, misiles, flares
Misil con lock	+ recompensa si <code>lock=1</code> y se lanza misil	Premia ataques precisos
Bloqueo con chaff	+ recompensa si se evita daño de misil	Premia defensa efectiva
Recursos usados	Penalización leve por chaff, burner, etc.	Evita gastar recursos innecesarios





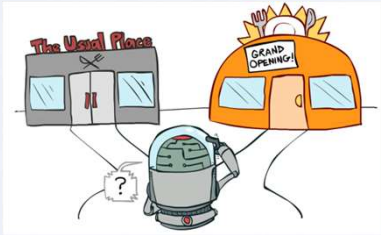
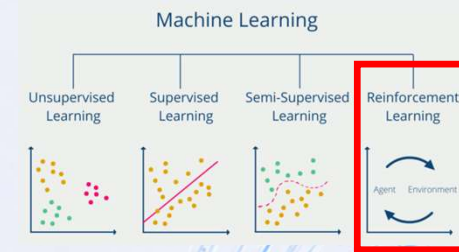
[ETAPA 6] Actualización de la política / función de valor

El agente **almacena la experiencia** :

(estado, acción, recompensa, siguiente_estado, done)

en un **buffer de replay**.

- Cada cierto número de pasos, se **entrena la red neuronal** para predecir el valor esperado de cada acción, minimizando la diferencia entre la predicción y la recompensa observada + valor futuro (target Q).
- Esto permite que el agente **aprenda gradualmente a elegir acciones que maximizan la recompensa acumulada**.



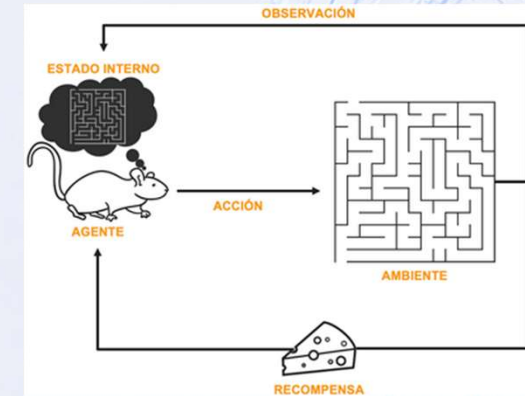
EXPLORAR O RECAUDAR?



RECOMPENSA



ENTRENAMIENTO





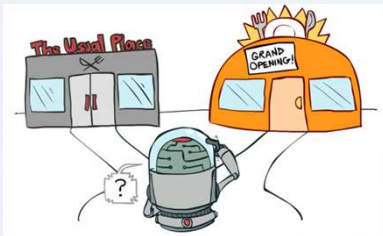
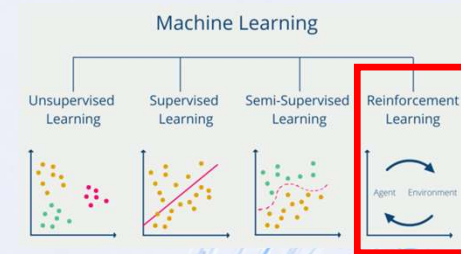
[ETAPA 7] Repetición del ciclo (episodios)

El entrenamiento ocurre en **múltiples episodios**:

- Cada episodio inicia con `reset()` del entorno.
- El agente interactúa hasta que `done = True`.
- La política se ajusta con las experiencias de cada episodio.

A medida que aumenta la experiencia:

- La exploración se reduce
- La política converge hacia un comportamiento **óptimo**.



EXPLORAR O RECAUDAR?



RECOMPENSA



ENTRENAMIENTO



```
01 import openai
02 from IPython import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10     respuesta = modelo.invoke(
11     ) prompt
12     return respuesta.content
13 )
14
15 if __name__ == "__main__":
16     print("¡Gracias!")
17     print("dudas, consultas...")
18     input("Presione una tecla para salir...")
```

> print("Gracias !")

> print("dudas, consultas...")

> input("Presione una tecla para salir...")



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383